

Tutorial - Semana 2, Encontro 2

Introdução

No Encontro 1, o Player (CharacterBody3D) foi montado dentro de `level_exploration.tscn` — sólido, alinhado ao chão, e já chamando `move_and_slide` a cada frame de física, mas ainda sem receber nenhuma velocidade. Este encontro resolve três problemas restantes. Primeiro, como traduzir uma tecla pressionada pelo jogador em movimento do personagem, sem que o código de gameplay precise saber qual tecla física foi usada — o Godot resolve isso com o **Input Map**, uma camada de abstração entre o dispositivo físico e a ação lógica do jogo. Segundo, como dar ao jogador uma visão do personagem em terceira pessoa que acompanhe o movimento sem atravessar paredes — o Godot resolve isso com **SpringArm3D + Camera3D**. Terceiro, como fazer o movimento seguir a direção da câmera, e não um eixo fixo do mundo — o padrão esperado em um jogo de aventura em terceira pessoa.

Este tutorial dá continuidade direta ao Encontro 1 — o Player já deve existir na Scene, parado, antes de começar.

Objetivos da semana

- Explicar Input Map e InputEvent como camada de desacoplamento entre dispositivo físico e ação lógica.
- Configurar um Input Map para movimentação.
- Conectar o Input Map ao `move_and_slide` do Player.
- Corrigir o `Chao` para ter colisão física real, entendendo a diferença entre `MeshInstance3D` (malha visual) e `StaticBody3D` (corpo físico estático).
- Configurar uma câmera em terceira pessoa (`SpringArm3D` + `Camera3D`) que acompanha o Player sem atravessar paredes.
- Ajustar o movimento para ser relativo à direção da câmera, e não a eixos fixos do mundo.

Resultado esperado ao final da semana

Ao final da Semana 2 (Encontros 1 e 2), cada estudante terá um Player (CharacterBody3D) movendo-se no nível de teste através de um Input Map próprio, com pelo menos uma Action adicional não demonstrada em aula, uma câmera em terceira pessoa acompanhando o personagem, e movimento relativo à direção da câmera. Este tutorial cobre apenas o **Encontro 2**: a configuração do Input Map, a conexão com a movimentação, a configuração da câmera, e o ajuste do movimento para ser relativo à câmera.

Pré-requisitos

- Player (CharacterBody3D) montado na Scene `level_exploration.tscn`, com `CollisionShape3D`, `Malha` e Orchestration `player.torch` chamando `move_and_slide` (ver Tutorial - Semana 2, Encontro 1).

Antes de começar

O que o estudante deverá possuir antes desta semana

- O projeto do Encontro 1, com o Player parado, porém fisicamente funcional, dentro de `level_exploration.tscn`.

Arquivos necessários

- Nenhum arquivo externo. O Input Map é configurado inteiramente dentro do Project Settings do Godot.

Assets utilizados

- Nenhum. Os pacotes Kenney começam a ser utilizados a partir da Semana 3.

Projeto esperado

- Projeto aberto no Godot 4.7, com o Player do Encontro 1 pronto para receber input.
- Orchestration `player.torch` já existente, com o evento PhysicsProcess chamando `move_and_slide`.

Imagem sugerida

Objetivo: mostrar a aba Input Map do Project Settings, com as Actions de movimentação já configuradas. Enquadramento: captura de tela da janela Project Settings, aba Input Map. Elementos importantes: lista de Actions (`move_forward`, `move_back`, `move_left`, `move_right`), teclas associadas a cada uma. Destaque: o botão "Add" usado para criar uma nova Action. Legenda sugerida: "Input Map do projeto com as Actions de movimentação configuradas."

Parte 1 — Input Map e InputEvent como camada de desacoplamento

Objetivo

Entender por que engines modernas inserem uma camada de abstração entre a tecla física pressionada e a ação lógica do jogo, antes de configurar qualquer Action no editor.

Conceito

Se o código de gameplay lesse diretamente "tecla W pressionada", qualquer remapeamento de controles ou suporte a um gamepad exigiria reescrever a lógica do jogo inteira, tecla por tecla. Por isso, engines modernas inserem uma camada de abstração entre o dispositivo físico e a ação lógica: o jogador aperta uma tecla, a engine traduz isso em uma **Action** nomeada (por exemplo, "mover para frente"), e é essa Action — não a tecla em si — que o código de gameplay consome.

No Godot, essa camada é o **Input Map**, configurado uma única vez no projeto (em **Project Settings > Input Map**), associando uma ou mais teclas/botões físicos a cada Action. Em tempo de execução, cada tecla pressionada gera um **InputEvent**, que o Godot já traduz automaticamente para a Action correspondente — o script ou a Orchestration não precisam tratar o evento bruto, apenas perguntar "a Action `move_forward` está pressionada agora?".

Passo a passo

1. Sem abrir o Project Settings ainda, discuta com a turma: se o código do jogo checasse diretamente a tecla `W`, o que aconteceria ao tentar oferecer suporte a um gamepad?
2. Abra **Project > Project Settings** e selecione a aba **Input Map**.
3. No campo de texto no topo, digite `move_forward` e clique em **Add**.
4. Com a Action `move_forward` criada, clique no ícone de `+` ao lado dela para adicionar um evento de tecla, pressione a tecla **W** e confirme.
5. Repita o processo para criar `move_back` (tecla **S**), `move_left` (tecla **A**) e `move_right` (tecla **D**).
6. Feche o Project Settings e reabra a Orchestration `player.torch` criada no Encontro 1.
7. Dentro do PhysicsProcess já existente, adicione um nó do Orchestrator que lê as quatro Actions e já retorna um vetor de direção 2D combinado (equivalente a `Input.get_vector()` em GDScript) — não é preciso ler cada Action separadamente e montar o vetor à mão.

8. Transforme esse vetor de direção em uma velocidade 3D, multiplicando por uma velocidade constante (por exemplo, 5.0) e atribuindo o resultado à propriedade `velocity` do `Player`.
9. Confirme visualmente no grafo que a leitura das quatro Actions ficou concentrada em um único nó — isso evita quatro nós de leitura separados e uma combinação manual, mais propensa a erro de sinal.
10. Confirme que a chamada a `move_and_slide`, já existente desde o Encontro 1, permanece após a atribuição de `velocity`.
11. Salve a Orchestration e a Scene, e pressione **Play Scene** (F6).
12. Utilize as teclas W, A, S e D para mover o Player pelo nível de teste, confirmando que ele desliza corretamente ao encostar em obstáculos.

Código de referência (GDScript)

Os passos 7–8 pedem para montar, no Orchestrator, a leitura das quatro Actions já combinadas em um vetor de direção (um único nó, equivalente a `Input.get_vector()`) e a atribuição do resultado à `velocity`. O trecho abaixo é o equivalente em GDScript — não é para copiar dentro de um editor de código, é uma referência para pensar em como organizar os nós antes de montar o grafo visual: um nó de leitura combinada de Actions, e em que ponto multiplicar o resultado pela velocidade constante.

```
extends CharacterBody3D

const SPEED := 5.0

func _physics_process(delta: float) -> void:
    var input_dir := Input.get_vector("move_left",
    "move_right", "move_forward", "move_back")

    velocity.x = input_dir.x * SPEED
    velocity.z = input_dir.y * SPEED

    move_and_slide()
```

Resultado esperado

O Player se move pelo nível de teste em resposta às teclas W, A, S e D, através de quatro Actions (`move_forward`, `move_back`, `move_left`, `move_right`) configuradas no Input Map do projeto e lidas pela Orchestration `player.torch`, que aplica a direção resultante à `velocity` do `CharacterBody3D` antes de chamar `move_and_slide`.

Nota: nesta etapa o movimento ainda é relativo aos eixos fixos do mundo — "frente" sempre anda na mesma direção, independentemente de para onde a câmera está olhando. Isso é temporário: a Parte 3 deste tutorial ajusta o movimento para ser relativo à câmera, depois que a Parte 2 configurar o `CameraPivot`.

Verificando

1. Abra **Project Settings > Input Map** e confirme que as quatro Actions existem, cada uma com a tecla correta associada.
2. Rode a Scene com F6 e mova o Player nas quatro direções, confirmando resposta imediata a cada tecla.
3. Encoste o Player em uma borda do `Chao` ou em outro obstáculo da cena e confirme que ele desliza ao longo da superfície, sem travar ou atravessar.

Problemas comuns

- Actions com nomes ambíguos ou duplicados no Input Map, causando comportamento inesperado: reforçar a convenção de nomes clara usada aqui (`move_forward` , `move_back` , `move_left` , `move_right`).
- Direção de movimento invertida (por exemplo, W move o Player para trás): checar a orientação do Node `Player` no Viewport antes de depurar a lógica de input — o eixo "frente" do `CharacterBody3D` depende de como ele foi rotacionado.
- Player não se move mesmo com o Input Map configurado: confirmar que a `velocity` está sendo atribuída antes da chamada a `move_and_slide` , e não depois — a ordem dentro do `PhysicsProcess` importa.
- Movimento "picotado" ou não suave: confirmar que a leitura de input e a chamada a `move_and_slide` estão de fato dentro do evento `PhysicsProcess`, e não em um evento de `Process` comum (por frame de renderização).

Boas práticas

- Nomear Actions por intenção do jogador (`move_forward`), nunca pela tecla física (`tecla_w`) — o nome da Action deve continuar fazendo sentido mesmo se o jogador remapear os controles.
- Centralizar toda leitura de input relacionada à movimentação dentro da `Orchestration` do próprio Player, evitando espalhar chamadas de Input Map por múltiplos Nodes.
- Testar cada Action isoladamente durante a configuração (uma tecla de cada vez) antes de combinar as quatro em um vetor de direção — isso facilita identificar qual Action está mal configurada em caso de erro.

Comparação com Unity

A Unity resolve o mesmo problema com o **Input System** (novo), usando **Action Maps** e um componente **Player Input** para conectar as Actions ao código — uma solução com mais camadas de configuração (Action Maps, Control Schemes, bindings por dispositivo) do que o Input Map do Godot. O Godot concentra tudo em um único Input Map global do projeto, mais simples de configurar para um caso como este, porém com menos granularidade nativa para múltiplos esquemas de controle simultâneos (por exemplo, dois jogadores locais com Action Maps distintos).

Parte 2 — Câmera em terceira pessoa (SpringArm3D + Camera3D)

Objetivo

Entender por que uma câmera presa atrás do personagem precisa de tratamento especial de colisão, e configurar essa câmera usando SpringArm3D + Camera3D, antes de o Player ficar de fato jogável em terceira pessoa.

Conceito

Uma câmera de terceira pessoa "ingênua" — apenas um Node posicionado atrás do personagem — atravessa paredes e cenário assim que o jogador encosta em um obstáculo, porque nada resolve a colisão entre a câmera e o mundo. Toda engine moderna que oferece câmera de terceira pessoa precisa resolver o mesmo problema: acompanhar o personagem, girar conforme o jogador movimenta o mouse, e nunca atravessar geometria.

No Godot, esse problema é resolvido pelo **SpringArm3D**: um Node que funciona como um "braço" entre um ponto de origem e a câmera, encurtando automaticamente sua extensão quando um raycast interno detecta que algo bloqueia a linha entre os dois pontos. A rotação da câmera é dividida em dois eixos, cada um resolvido por um Node diferente: o giro horizontal (yaw), que gira o personagem em torno de si, fica em um `Node3D` pivô; o giro vertical (pitch), que olha para cima e para baixo, fica no próprio SpringArm3D — separar os dois evita que a câmera vire de cabeça para baixo, já que o pitch pode ser limitado (clamp) independentemente do yaw.

O SpringArm3D só consegue encurtar sua extensão se houver, de fato, um corpo físico contra o qual colidir. E aqui aparece um problema arrastado desde a Semana 1: o `Chao` foi montado como um `MeshInstance3D` puro — apenas malha visual, sem nenhum volume físico. O Player "parecia" parado sobre o `Chao` só porque foi posicionado manualmente ali; nada nunca colidiu de verdade com

ele. Este é o momento certo para corrigir isso e, de quebra, fixar uma distinção fundamental do Godot: **MeshInstance3D** é puramente visual (o que a câmera renderiza), enquanto **StaticBody3D** é o Node de física para corpos sólidos que nunca se movem sozinhos (cenário, paredes, chão) — é ele quem entrega volume físico real para outros sistemas (CharacterBody3D, SpringArm3D, RayCast3D) colidirem contra.

Passo a passo

1. Sem abrir o editor ainda, discuta com a turma: se a câmera fosse apenas um Node3D fixo atrás do Player, sem nenhum tratamento, o que aconteceria ao encostar o personagem em uma parede?
2. Reabra a Scene `level_exploration.tscn` com o Player já controlável do Passo a passo anterior.
3. Selecione o Node `Chao` no painel Scene e observe, no Inspector, que seu tipo é `MeshInstance3D` — puramente visual, sem volume físico. Até agora, nada no nível colide de fato com ele.
4. Renomeie temporariamente `Chao` para `Malha` (ele continua sendo o mesmo Node visual, só muda de nome).
5. Clique com o botão direito sobre `NivelTeste` e adicione um novo Node filho do tipo **StaticBody3D**, renomeando-o para `Chao`.
6. No painel Scene, arraste `Malha` para dentro do novo `Chao` (StaticBody3D), tornando-o filho dele.
7. Com `Chao` (StaticBody3D) selecionado, adicione um Node filho **CollisionShape3D** e, no Inspector, atribua uma Shape (por exemplo, uma `BoxShape3D`) do mesmo tamanho da `Malha`.
8. Salve a Scene (**Ctrl+S**) e confirme, no painel Scene, que a hierarquia agora é `NivelTeste > Chao` (StaticBody3D) > `Malha` (MeshInstance3D) + `CollisionShape3D`.
9. Selecione o Node `Player` e adicione um Node filho `Node3D`, renomeado para `CameraPivot`, posicionado na altura aproximada dos ombros do personagem.
10. Dentro de `CameraPivot`, adicione um Node `SpringArm3D`.
11. No Inspector do `SpringArm3D`, ajuste **Spring Length** (por exemplo, 4.0) e configure a **Collision Mask** para detectar apenas as camadas de colisão do cenário (nunca a camada do próprio Player).
12. Dentro do `SpringArm3D`, adicione um Node `Camera3D` como filho — ele herda automaticamente a posição resolvida pelo SpringArm3D a cada frame.
13. Confirme que o `Camera3D` está marcado como **Current** (ou que é a única câmera ativa na Scene).
14. Reabra a Orchestration `player.torch` e, no evento Ready, adicione a lógica que captura o mouse (equivalente a `Input.mouse_mode = Input.MOUSE_MODE_CAPTURED` em GDScript).
15. Adicione um nó de evento de Input não tratado (Unhandled Input) que só reage a eventos de movimento do mouse (`InputEventMouseMotion`).

16. Dentro desse evento, rotacione o `CameraPivot` no eixo Y (yaw), proporcionalmente ao deslocamento horizontal do mouse.
17. Rotacione o `SpringArm3D` no eixo X (pitch), proporcionalmente ao deslocamento vertical do mouse, limitando (clamp) o valor para impedir que a câmera vire de cabeça para baixo.
18. Salve a Orchestration e a Scene, e pressione **Play Scene** (F6).
19. Movimente o mouse para confirmar que a câmera gira suavemente ao redor do Player, e encoste o personagem em uma parede do nível de teste para confirmar que a câmera se aproxima automaticamente, sem atravessar a geometria.

Código de referência (GDScript)

Os passos 8–11 pedem para montar, no Orchestrator, a captura do mouse e a rotação de yaw/pitch em resposta ao movimento do mouse. O trecho abaixo é o equivalente em GDScript — não é para copiar dentro de um editor de código, é uma referência para pensar em como organizar os nós antes de montar o grafo visual: um nó de evento Ready capturando o mouse, e um nó de evento Unhandled Input filtrando `InputEventMouseMotion` e rotacionando `CameraPivot` e `SpringArm3D`.

```
extends CharacterBody3D

@onready var camera_pivot: Node3D = $CameraPivot
@onready var spring_arm: SpringArm3D =
    $CameraPivot/SpringArm3D

const MOUSE_SENSITIVITY := 0.003
const PITCH_MIN := deg_to_rad(-40.0)
const PITCH_MAX := deg_to_rad(60.0)

func _ready() -> void:
    Input.mouse_mode = Input.MOUSE_MODE_CAPTURED

func _unhandled_input(event: InputEvent) -> void:
    if event is InputEventMouseMotion:
        camera_pivot.rotate_y(-event.relative.x *
MOUSE_SENSITIVITY)
        spring_arm.rotation.x = clamp(
            spring_arm.rotation.x - event.relative.y *
MOUSE_SENSITIVITY,
            PITCH_MIN,
            PITCH_MAX
        )
```


Resultado esperado

O `Chao`, que até este encontro era apenas uma malha visual (`MeshInstance3D`), passa a ser um corpo físico estático (`StaticBody3D` + `CollisionShape3D`). A câmera acompanha o Player em terceira pessoa, gira suavemente com o movimento do mouse (yaw no `CameraPivot`, pitch com clamp no `SpringArm3D`) e nunca atravessa paredes ou obstáculos do nível de teste, graças à colisão automática resolvida pelo `SpringArm3D` contra esse novo corpo físico.

Verificando

1. Confirme, no painel Scene, que `Chao` é um `StaticBody3D` com `Malha` (`MeshInstance3D`) e `CollisionShape3D` como filhos.
2. Gire a câmera 360° ao redor do Player com o mouse e confirme que não há travamentos nem giros bruscos.
3. Tente olhar bem para cima e para baixo e confirme que o clamp de pitch impede a câmera de virar de cabeça para baixo.
4. Encoste o Player em uma parede ou canto do nível de teste e confirme que a câmera se aproxima automaticamente, sem atravessar a geometria.

Problemas comuns

- Câmera atravessa o `Chao` mesmo depois de configurar a Collision Mask: confirmar que `Chao` foi de fato convertido em `StaticBody3D` com `CollisionShape3D` — um `MeshInstance3D` sozinho nunca gera colisão, não importa a Collision Mask configurada no `SpringArm3D`.
- Câmera atravessando paredes: confirmar que a Collision Mask do `SpringArm3D` inclui a camada de colisão do cenário (`Chao` e demais obstáculos).
- Mouse não é capturado, cursor continua visível: confirmar que `Input.mouse_mode = Input.MOUSE_MODE_CAPTURED` está sendo chamado no evento Ready da Orchestration.
- Câmera vira de cabeça para baixo: confirmar que o clamp do pitch (rotação em X do `SpringArm3D`) está sendo aplicado antes de atribuir o valor.
- Câmera colidindo com o próprio Player: revisar a Collision Mask do `SpringArm3D`, removendo a camada de colisão usada pelo `CollisionShape3D` do Player.

Boas práticas

- Tratar todo cenário estático (chão, paredes, plataformas) sempre como `StaticBody3D` + `CollisionShape3D` + `MeshInstance3D` — o mesmo princípio de separar física e visual já aplicado ao Player no Encontro 1, agora estendido a qualquer objeto sólido do nível.

- Separar yaw (`CameraPivot`) e pitch (`SpringArm3D`) em Nodes distintos — misturar as duas rotações no mesmo Node dificulta o clamp do pitch.
- Sempre limitar (clamp) o pitch da câmera — sem isso, a câmera pode girar 360° verticalmente e quebrar a legibilidade da cena.
- Ajustar a Collision Mask do `SpringArm3D` para detectar apenas o cenário, nunca o próprio Player.
- Centralizar a lógica de câmera na mesma Orchestration do Player, evitando espalhar Nodes de câmera avulsos pela Scene.

Comparação com Unity

A Unity resolve o mesmo problema tipicamente com o pacote **Cinemachine**: um **Cinemachine Virtual Camera** cuida do enquadramento e do follow/look-at, e um **Cinemachine Collider** resolve a colisão da câmera com o cenário — conceitualmente equivalente ao `SpringArm3D` do Godot. A diferença arquitetural: no Godot, o `SpringArm3D` é um Node nativo simples, disponível em qualquer projeto sem addons; na Unity, o comportamento equivalente depende de um pacote separado (Cinemachine), com mais componentes e configuração (Virtual Camera, Collider, Confiner, Brain), mas também com recursos mais avançados prontos (blends entre câmeras, damping refinado).

Parte 3 — Movimento relativo à câmera

Objetivo

Corrigir o movimento construído na Parte 1 para que ele siga a direção da câmera configurada na Parte 2, em vez de eixos fixos do mundo — o padrão esperado em um jogo de aventura em terceira pessoa.

Conceito

Com o movimento da Parte 1, "andar para frente" sempre desloca o Player no mesmo eixo do mundo, não importa para onde a câmera esteja apontando — um esquema de controle chamado *tank controls* (comum em jogos de survival horror clássicos, como os primeiros Resident Evil). Jogos de aventura em terceira pessoa modernos (Zelda, Dark Souls, a própria TPS Demo oficial do Godot) usam **movimento relativo à câmera**: pressionar "frente" move o personagem na direção para onde a câmera está olhando, não em um eixo fixo.

A correção não muda o Input Map nem a leitura das Actions — muda apenas como o vetor de direção 2D é convertido em uma direção 3D. Em vez de aplicar `input_dir.x` / `input_dir.y` direto aos

eixos globais X/Z, o vetor precisa ser rotacionado pela orientação horizontal (yaw) da câmera, ou seja, pela rotação do `CameraPivot`. Como o `CameraPivot` só gira no eixo Y (o pitch fica isolado no `SpringArm3D`, como visto na Parte 2), sua Basis representa exatamente a direção horizontal para onde o jogador está olhando — sem nenhuma inclinação vertical contaminando o movimento.

Passo a passo

1. Reabra a Orchestration `player.torch` e localize os nós que calculam `input_dir` e atribuem `velocity.x / velocity.z`, construídos na Parte 1.
2. Entre a combinação do vetor 2D e a atribuição à `velocity`, adicione um nó que monte um `Vector3` a partir de `input_dir` (`Vector3(input_dir.x, 0, input_dir.y)`), igual ao que já existia.
3. Adicione um nó que leia a Transform (Basis) do `CameraPivot` — não mais a do próprio `Player`.
4. Multiplique esse `Vector3` pela Basis do `CameraPivot` (nó `A * B`, como no restante do grafo) e normalize o resultado.
5. Substitua os nós que atribuíam `velocity.x / velocity.z` diretamente a partir de `input_dir` para usarem, em vez disso, o resultado normalizado do passo 4, multiplicado pela `SPEED`.
6. Salve a Orchestration e a Scene, e pressione **Play Scene** (F6).
7. Gire a câmera com o mouse sem se mover, depois pressione "frente" (W) e confirme que o Player anda na direção para onde a câmera está olhando, não mais em um eixo fixo do mundo.

Código de referência (GDScript)

Este é o script completo da Orchestration `player.torch` ao final da Semana 2 — junta o movimento da Parte 1, a câmera da Parte 2, e a correção desta Parte 3. Não é para copiar dentro do editor de código; é a referência para pensar em como organizar o grafo completo do Orchestrator.

```
extends CharacterBody3D

const SPEED := 5.0
const MOUSE_SENSITIVITY := 0.003
const PITCH_MIN := deg_to_rad(-40.0)
const PITCH_MAX := deg_to_rad(60.0)

@onready var camera_pivot: Node3D = $CameraPivot
@onready var spring_arm: SpringArm3D =
    $CameraPivot/SpringArm3D

func _ready() -> void:
```

```

Input.mouse_mode = Input.MOUSE_MODE_CAPTURED

func _unhandled_input(event: InputEvent) -> void:
    if event is InputEventMouseMotion:
        camera_pivot.rotate_y(-event.relative.x *
MOUSE_SENSITIVITY)
        spring_arm.rotation.x = clamp(
            spring_arm.rotation.x - event.relative.y *
MOUSE_SENSITIVITY,
            PITCH_MIN,
            PITCH_MAX
        )

func _physics_process(delta: float) -> void:
    var input_dir := Input.get_vector("move_left",
"move_right", "move_forward", "move_back")

    var direction := (camera_pivot.transform.basis *
Vector3(input_dir.x, 0, input_dir.y)).normalized()
    velocity.x = direction.x * SPEED
    velocity.z = direction.z * SPEED

    move_and_slide()

```

Resultado esperado

O Player se move na direção para onde a câmera está olhando, não mais em eixos fixos do mundo. Girar a câmera com o mouse muda imediatamente a direção do movimento nas teclas W, A, S e D, sem inclinar o movimento verticalmente mesmo quando a câmera está olhando para cima ou para baixo.

Verificando

1. Gire a câmera 180° com o mouse, mantendo o Player parado, e então pressione "frente" (W): confirme que o Player anda para o novo lado para onde a câmera aponta.
2. Incline a câmera para cima ou para baixo (pitch) e confirme que o movimento continua horizontal, sem o Player "afundar" ou "subir".
3. Combine movimento e rotação de câmera simultaneamente e confirme que não há travamentos ou movimento invertido.

Problemas comuns

- Movimento continua fixo no eixo do mundo: confirmar que a Basis usada no cálculo é a do `CameraPivot`, não a do `Player`.
- Movimento inclinado para cima/baixo ao olhar a câmera para cima/baixo: confirmar que a Basis usada é a do `CameraPivot` (só yaw), e não a do `SpringArm3D` (que também tem pitch).
- Player anda na direção contrária à esperada: revisar o sinal usado na rotação do `CameraPivot` (Passo 10 da Parte 2) antes de mexer na lógica de movimento.

Boas práticas

- Nunca usar a Basis do `SpringArm3D` (que contém pitch) para calcular a direção de movimento — isso inclinaria o deslocamento do Player para cima/baixo junto com a câmera.
- Normalizar o vetor de direção resultante antes de multiplicar pela `SPEED`, evitando movimento diagonal mais rápido que o movimento reto.
- Manter a leitura de Actions (Parte 1) e a leitura da câmera (Parte 2) como blocos separados e nomeados dentro do mesmo grafo, mesmo que agora se conectem — facilita revisar cada bloco isoladamente.

Comparação com Unity

O mesmo ajuste na Unity é feito projetando os vetores `Camera.main.transform.forward` e `Camera.main.transform.right` no plano horizontal (zerando o componente Y) e combinando-os com o input, em vez de usar os eixos globais `Vector3.forward` / `Vector3.right` diretamente — o mesmo princípio do Godot, de usar a orientação da câmera (não do personagem nem do mundo) como referência para o movimento.

Ao final da semana

Ao final da Semana 2 (Encontros 1 e 2), o projeto do Vertical Slice deve conter:

- O Player (`CharacterBody3D`) montado no Encontro 1, com `CollisionShape3D` e `Malha` alinhados.
- O `Chao` convertido de `MeshInstance3D` solto para `StaticBody3D` + `CollisionShape3D` + `Malha` (`MeshInstance3D`), com colisão física real.
- Um Input Map do projeto com as Actions `move_forward`, `move_back`, `move_left`, `move_right`, mais uma Action adicional criada no desafio deste encontro.

- Uma câmera em terceira pessoa (`CameraPivot` + `SpringArm3D` + `Camera3D`) filha do Player, acompanhando o personagem sem atravessar paredes.
- A Orchestration `player.torch` lendo o Input Map e aplicando a direção resultante — já rotacionada pela Basis do `CameraPivot` — a `move_and_slide` , tornando o Player efetivamente controlável e com movimento relativo à câmera.

Segundo o `PROJECT_ARCHITECTURE.md` (seção 6, Módulo 1), este resultado corresponde à conclusão dos itens "Player (locomção)", "Câmera" e "Input do jogador", pré-requisitos diretos para a Cena de teste (graybox) e para a Renderização/build, que serão construídas na Semana 3, encerrando o Módulo 1.

Desafio

Adicione uma nova Action ao Input Map, não demonstrada neste tutorial — correr, agachar ou pular —, com liberdade de implementação (por exemplo, correr como multiplicador de velocidade aplicado à `velocity` , ou pular como um impulso vertical simples somado a `velocity.y`). Não há solução única.

Checklist

- ☐ `Chao` convertido em `StaticBody3D` , com `Malha` (`MeshInstance3D`) e `CollisionShape3D` como filhos
- ☐ Input Map do projeto com as Actions `move_forward` , `move_back` , `move_left` , `move_right`
- ☐ Orchestration `player.torch` lendo as quatro Actions já combinadas em um vetor de direção (equivalente a `Input.get_vector()`)
- ☐ `velocity` do Player atribuída antes da chamada a `move_and_slide`
- ☐ Player se move nas quatro direções e desliza corretamente ao encostar em obstáculos
- ☐ Scene testada com F6, sem erros
- ☐ Action adicional do desafio (correr, agachar ou pular) criada e funcional
- ☐ `CameraPivot` (`Node3D`) + `SpringArm3D` + `Camera3D` configurados como filhos do Player
- ☐ Câmera gira com o mouse (yaw no `CameraPivot`, pitch com clamp no `SpringArm3D`) sem atravessar paredes
- ☐ Movimento usa a Basis do `CameraPivot` (não eixos fixos do mundo) — "frente" segue a direção da câmera

Glossário

- **Input Map:** configuração global do projeto que associa teclas/botões físicos a Actions nomeadas.
- **Action:** nome lógico de uma intenção do jogador (ex.: `move_forward`), desacoplado da tecla física usada para acioná-la.
- **InputEvent:** evento gerado pelo Godot a cada interação do jogador com um dispositivo físico, traduzido automaticamente para a Action correspondente.
- **velocity:** propriedade do `CharacterBody3D` que define a direção e intensidade do movimento antes da chamada a `move_and_slide`.
- **PhysicsProcess:** evento chamado a cada frame de física, ponto correto para ler input e mover o `CharacterBody3D`.
- **SpringArm3D:** Node que resolve automaticamente a colisão da câmera com o cenário, encurtando a distância entre a origem e a ponta do braço quando algo bloqueia a linha entre os dois pontos.
- **StaticBody3D:** Node de física para corpos sólidos que nunca se movem sozinhos (cenário, paredes, chão) — fornece colisão real contra `CharacterBody3D`, `SpringArm3D` e outros sistemas de física, ao contrário de um `MeshInstance3D`, que é puramente visual.
- **CameraPivot:** convenção de nome para um `Node3D` usado como eixo de rotação horizontal (yaw) da câmera, filho do `Player`.
- **Yaw / Pitch:** rotação horizontal e vertical da câmera, respectivamente — no Godot, resolvidas em Nodes separados (`CameraPivot` e `SpringArm3D`) para facilitar o clamp do pitch.
- **MOUSE_MODE_CAPTURED:** modo de captura do mouse do Godot que trava e oculta o cursor, entregando apenas o delta de movimento via `InputEventMouseMotion`.

Referências

- Godot Documentation — Inputs:
<https://docs.godotengine.org/en/stable/tutorials/inputs/index.html>
- Godot Documentation — Physics — `CharacterBody3D`:
https://docs.godotengine.org/en/stable/classes/class_characterbody3d.html
- Godot Documentation — `SpringArm3D`:
https://docs.godotengine.org/en/stable/classes/class_springarm3d.html
- Godot Documentation — `Camera3D`:
https://docs.godotengine.org/en/stable/classes/class_camera3d.html
- Orchestrator — Documentação oficial: <https://orchestrator.cratercrash.space/>
- Unity Manual (consulta comparativa) — Input System:
<https://docs.unity3d.com/Manual/com.unity.inputsystem.html>
- Unity — Cinemachine (consulta comparativa):
<https://docs.unity3d.com/Packages/com.unity.cinemachine@latest>
- GDQuest: <https://www.gdquest.com/>